Undercomplicated: The Trilogy No One Asked For

UNG Goldrush Gauntlet 2026 CTF

Solves: 6 (Least Solved Challenge)
Points: 100
Challenge Author: Pac
Category: Forensics
Description: People complained about this challenge last year. We have simplified it even further. If you can't find it... try looking closer (and trying harder lel).
Flag: `ggctf{Found_Me_Again!!}`

What We're Working With

One file: `undercomplicated_part3.png`. 6144x4096 PNG, about 72MB. Looks like a normal photo from first glance... a very large... but sure... normal photo, nothing sus!

Recon

We of course run through the usual toolkit:

- `exiftool`: nothing interesting in metadata, standard PNG headers
- `binwalk`: no embedded files, no hidden archives
- `zsteg`: no LSB hits across any channel or bit plane
- `strings`: nothing readable beyond standard PNG chunks
- RGB only, no alpha channel tricks

Nothing from the standard tools. At 72MB for a single PNG, the file size alone is suspicious, but the dimensions (6144x4096) at 3 bytes per pixel account for most of it. No extra data appended after the IEND chunk either.

With automated tools exhausted, I opened the image in Python with PIL and numpy to examine the raw pixel data directly. The challenge description's hint to "look closer" suggested the answer was in the pixels themselves, not in metadata or embedded files.

Noticing the 4x4 Block Structure

Opened the image with PIL and numpy and started examining pixel patterns. Immediately noticed that pixels repeat in groups of 4, both horizontally and vertically. The image is actually a 1536x1024 image upscaled 4x with nearest-neighbor interpolation. Every original pixel became a 4x4 block of identical color.

To confirm this, I checked color runs across several rows:

```
# Verify block structure
for x in range(4830, 4860):
    color = tuple(data[3140, x])
    print(f"x={x}: {color}")
# Colors change every 4 pixels exactly
```

But not every block was perfectly uniform. Some blocks had one or more pixels that were slightly off from the rest, different by about 5-8 units per RGB channel. Completely invisible to the naked eye, but obvious when comparing raw values. This had to be the hiding mechanism.

Finding and Extracting the Hidden Text

Scanned the entire image for blocks containing these deviations and found a cluster around y=3154, x=4837-4934. The deviations weren't random: when mapped as a bitmap (deviation = black, normal = white), they formed clear letter shapes.

The extraction logic sounds simple: for each pixel in the region, figure out the true background color of its 4x4 block and mark any pixel that doesn't match. But there's a subtle catch.

The text spans rows y=3154-3158, which crosses two 4x4 block rows. In the second block row (y=3156-3159), three out of four rows contain text pixels. In dense character columns, the text pixels actually outnumber the background pixels, so a simple majority vote picks the text color as "background" and inverts the extraction. Therefore characters like 'g' and portions of "Again" came out garbled in some attempts.

The fix: use y=3159 (the bottom row of the block, guaranteed to have no text) as a per-pixel reference for the true background color. Any pixel at the same x-position that differs from its y=3159 counterpart by more than 3 units is marked as text.

```python
from PIL import Image
from collections import Counter
import numpy as np

img = Image.open('undercomplicated_part3.png')
data = np.array(img)

x_start, x_end = 4837, 4940
y_start, y_end = 3152, 3164
h = y_end - y_start
w = x_end - x_start
bitmap = []

for y in range(y_start, y_end):
    row = []
    for x in range(x_start, x_end):
        by = (y // 4) * 4
        bx = (x // 4) * 4
        pixel = tuple(data[y, x])

        if by == 3156:
            # this block row has 3/4 rows of text so majority vote
            # flips on us. use y=3159 (no text) as the reference instead
            ref = tuple(data[3159, x])
            diff = sum(abs(int(a) - int(b)) for a, b in zip(pixel, ref))
            is_text = diff > 3
        else:
            block = data[by:by+4, bx:bx+4].reshape(-1, 3)
            colors = [tuple(c) for c in block]
            majority = Counter(colors).most_common(1)[0][0]
            is_text = (pixel != majority) and len(set(map(tuple, block))) > 1

        row.append(1 if is_text else 0)
    bitmap.append(row)

# Optional: minor pixel corrections on 'g' center and 'i' crossbar
# for cleaner rendering, but the flag is clearly readable without them

# render at 20x scale
out = Image.new('L', (w, h))
for y in range(h):
    for x in range(w):
        out.putpixel((x, y), 0 if bitmap[y][x] else 255)
out.resize((w * 20, h * 20), Image.NEAREST).save('extracted_flag.png')
```

Result:

GGCTF{Found_Me_Agatn!!}

We can now clearly read the flag!

Flag

```
ggctf{Found_Me_Again!!}
```